AI Large Language Model General Course

Large Language Models Made Simple &
Your First Local Deployment with FlagOS

Have you ever used the following?





KIMI









Conversation Writing Q&A

But they are just AI that can "converse"

Case 1: Late-Night Office Dialogue Assistant

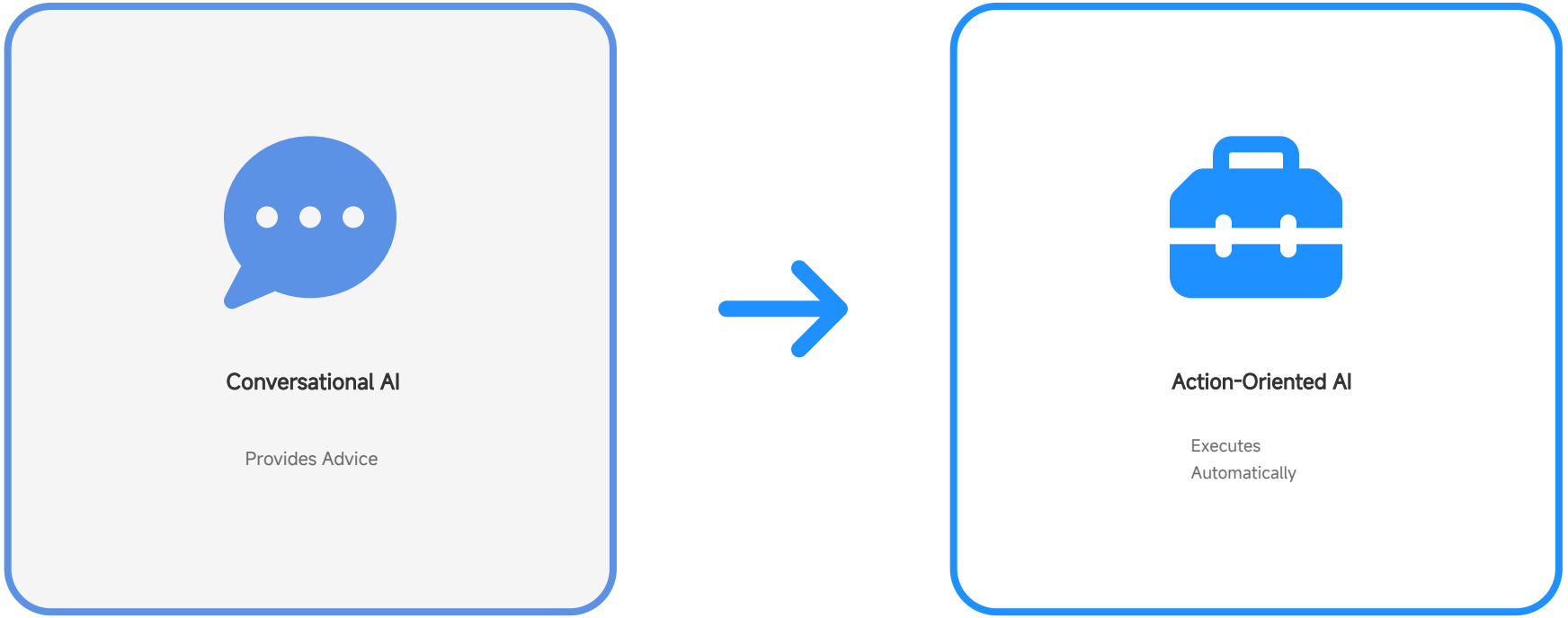
2 AM

"Help me generate a complete report package, including PPT outline and data analysis"

10 minutes

Material generation completed

Limitations of Conversation: Can Only Give Advice, Cannot Take Action



From "Asking AI How to Write an Email"

to "AI Itself Sends Emails, Books Meeting Rooms, Organizes Data, ..."

Case 2: Intelligent Agents Represented by OpenClaw



Step 1: Search

Find relevant materials



Step 2: Filter

Extract key information



Step 3: Generate

Write complete report



Step 4: Analyze

Data visualization



Step 5: Send

Automatic email notification



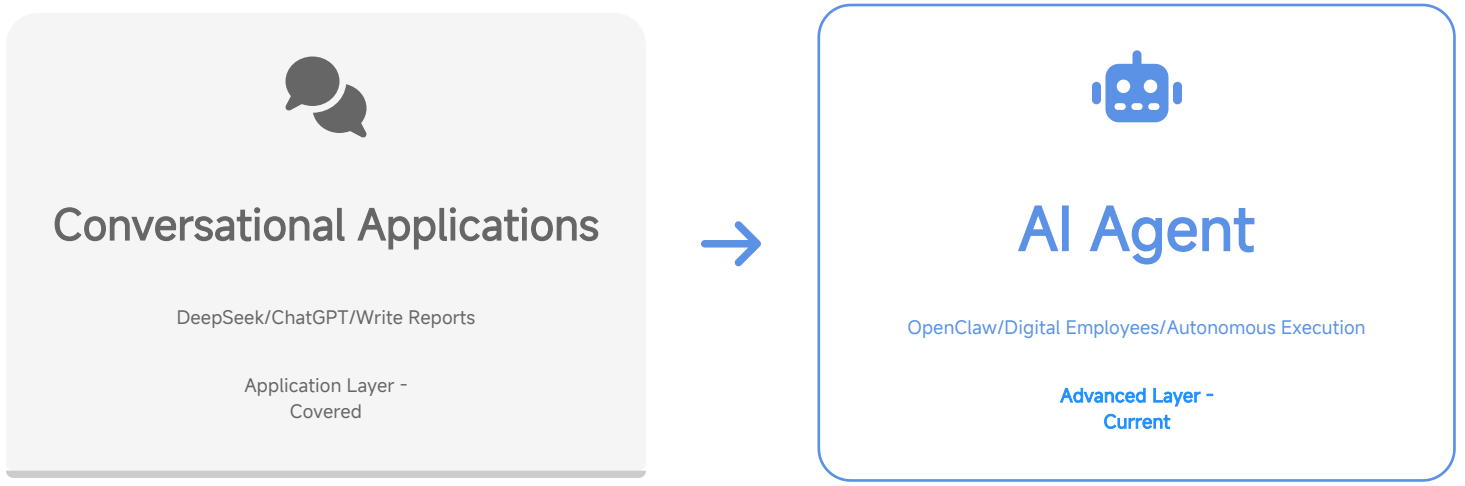
Step 6: Complete

Task closed-loop

> "Help me research the latest AI industry trends, generate a report and send it to the team"

📖 **AI Agent (Intelligent Agent)** | AI that can autonomously execute tasks - not just chatting, but capable of calling tools to complete actions

AI's Brain



Whether conversational or action-oriented, behind them all is the same"Brain"

Large
Model

Course Structure

1

"Omnipotent" AI

The Era of Large Models Has Arrived

2

How Does AI Actually Work?

Large Model Inference

3

How to Become an "Omnipotent" AI?

Large Model Training

4

Computing Power Behind Large Models

Open Source AI System Software Stack: FlagOS

5

Hands-on Demo

How to Run Large Models Locally with FlagOS

Appendix

Reference and Learning Resources

Acknowledgments

Thanks to Those Who Supported This Course

PART 1

"Omnipotent" AI

The Era of Large Models Has Arrived

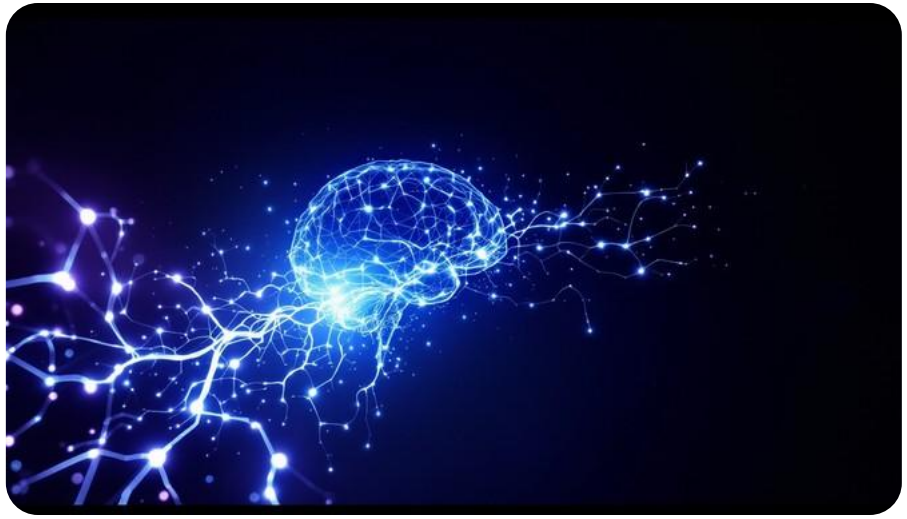
What is a Large Model?

Imagine a super library that stores almost all the knowledge and information in human history.

A large model is like the librarian of this library, who not only remembers all the content but also understands the connections and patterns among them.

Parameter Scale for **Deepseek-V3**

671 Billion



Parameters | Adjustable variables in the model, similar to the connection strength of neurons in the brain

The Magic of Generative AI



Text Generation

Articles, code, poetry, emails...



Image Generation

Painting, design, photography, illustration...



Audio Generation

Music, voiceover, sound effects, speech...



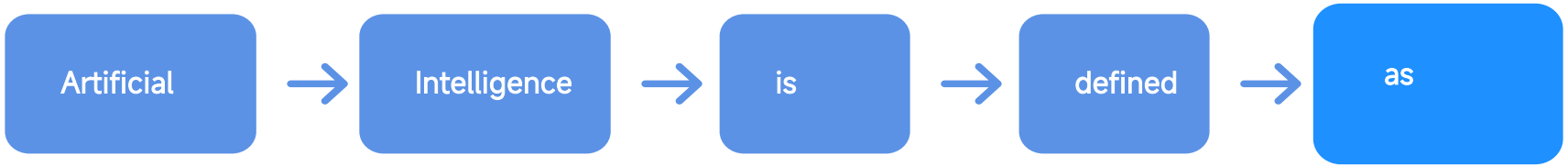
Video Generation

Animation, short films, special effects, editing...

📖**Multimodal** | Capable of simultaneously understanding and generating text, images, audio, video, and other types of content

Core Principle: The Prediction Game

The core capability of large models is "word chaining" — predicting the next most likely word



Candidate 1 ✓	Candidate 2	Candidate 3
computer	a type of	by
Probability: 45%	Probability: 30%	**Probability: 25%**

📖Autoregressive Generation | Predicting the next word based on previously generated content

Long-Distance Dependency Understanding

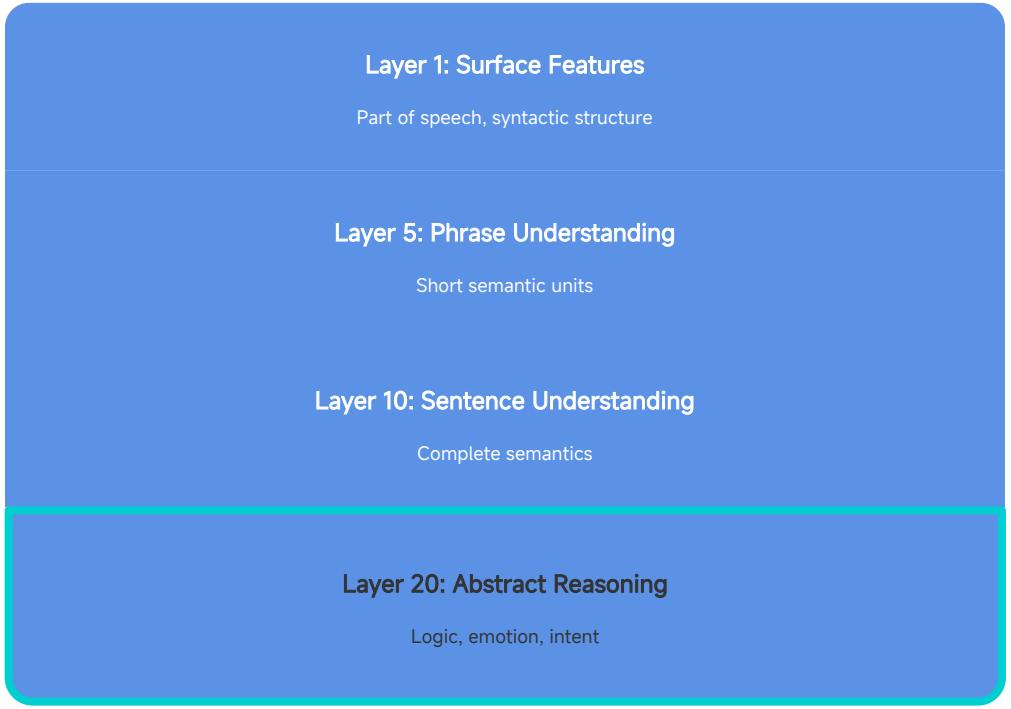
He raised a cat, and that cat, during his loneliest days when he worked overtime every night until late and was almost depressed, stayed by his side quietly the whole time, and was really adorable.



The model can understand long-distance semantic relationships

📖 **Self-Attention Mechanism**: Allows the model to automatically associate context words when processing each word

Multi-Layer Stacking: Transformer Architecture



Each layer extracts higher-level features based on the previous layer

📖**Transformer** | The core architecture of large models, where each layer consists of multi-head attention and feed-forward networks stacked together

How Do Large Models Work?

We just learned about the **capabilities** of large models:

- Text Generation
- Multimodal Capabilities
- Conversational Applications
- Agent Workflows



So, how exactly do large models **work**? how do large models actually operate

The process of applying large models is essentially the process of **large model inference**

Let's **break it down** with practical examples

PART 2

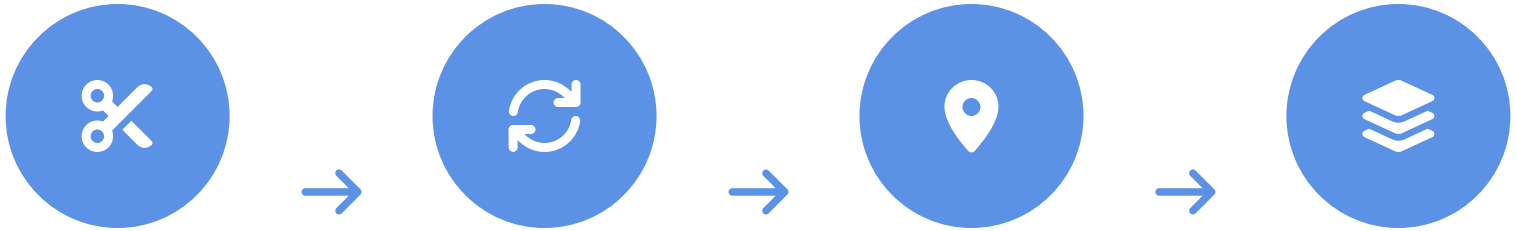
How Does AI Actually Do It?

Large Model Inference

Overview of Large Model Inference Process



Step 1 Input Processing: Text → Numbers



1.1 Tokenization

Split Text

1.2 Embedding

Convert to Vectors

1.3 Positional Encoding

Add Position

1.4 Final Representation

Add Vectors

Unified Example:"What is AI?"

Step 1.1 Tokenization

Split sentences into Tokens and look up the vocabulary to get Token IDs

"What is AI?" →

What	is	AI	?
ID: 1234	ID: 567	ID: 8901	ID: 234

Mainstream Vocabulary Size

100K+

Token

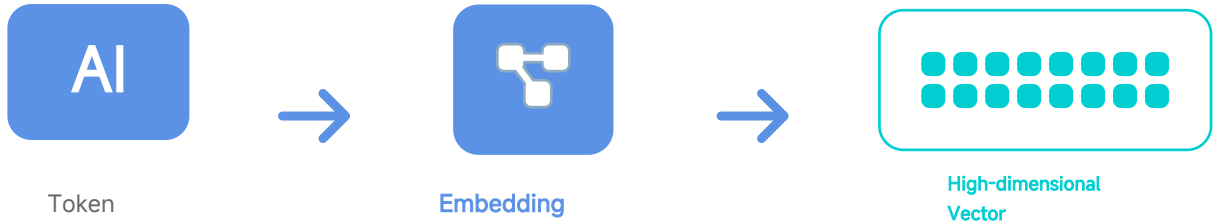
Each Token corresponds to a unique numeric ID

Try tokenization on this website:
https://tiktokenizer.vercel.app/
Enter any text and you will see how it is split into tokens

📖**Token** | The basic unit of text processing in large models, which can be a word, subword, or character; **Tokenization** | The process of splitting raw text into Token sequences; **BBPE (Byte-Level Byte Pair Encoding)** | A tokenization method that applies the BPE algorithm at the byte level, effectively handling multiple languages and special characters

Step 1.2 Vectorization

Map Tokens to high-dimensional vectors to capture semantic features



The higher the dimension, the greater the model's capacity to capture subtle semantic features, but the higher the computational cost as well

📖**Embedding** | A technique that maps Tokens to numerical vectors; **Vector** | An ordered set of numbers representing features

Step 1.3 Positional Encoding

Provides word order information to the model, distinguishing identical words at different positions

"What is AI?"

What	Position 1
is	Position 2
AI	Position 3

"Is What AI"

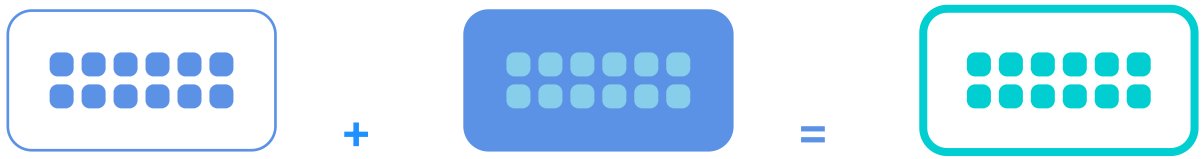
Is	Position 1
What	Position 2
AI	Position 3

Word Vector + Position Vector = Final Vector

📖**Positional Encoding** | A technique that provides word order information to the model

Step 1.4 Final Representation

Fusing semantic features with positional information to form the final input representation



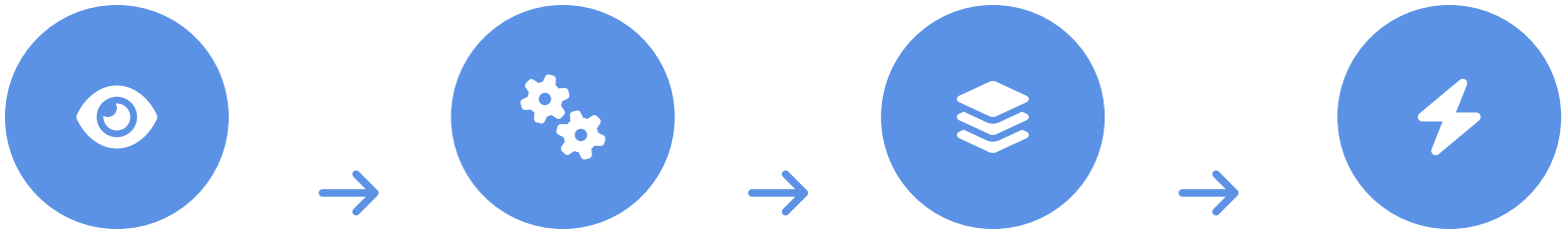
Word Vector
Semantic Features

Positional Vector
Sequential Information

Final Vector
Complete Representation

Step 1 Complete! Input successfully converted to machine-understandable numerical representation

Step 2 Layer-by-layer Computation: Understanding Semantics



2.1 Multi-angle Understanding

2.2 Deep Processing

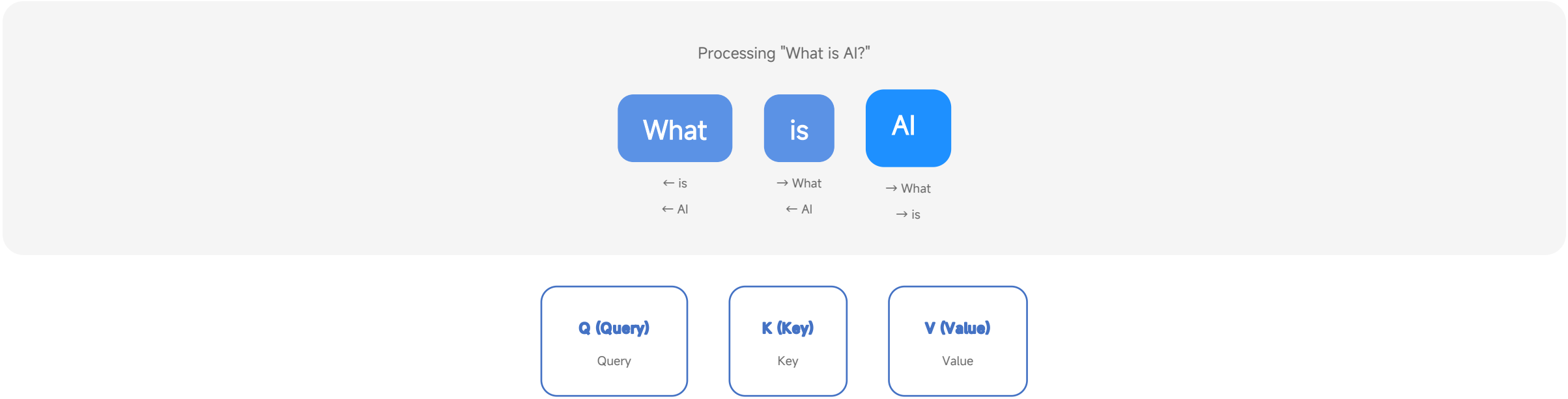
2.3 Progressive Layering

2.4 Cache Acceleration

Through multi-layer neural network computation, the model progressively understands the input's semantics, grammar and contextual relationships

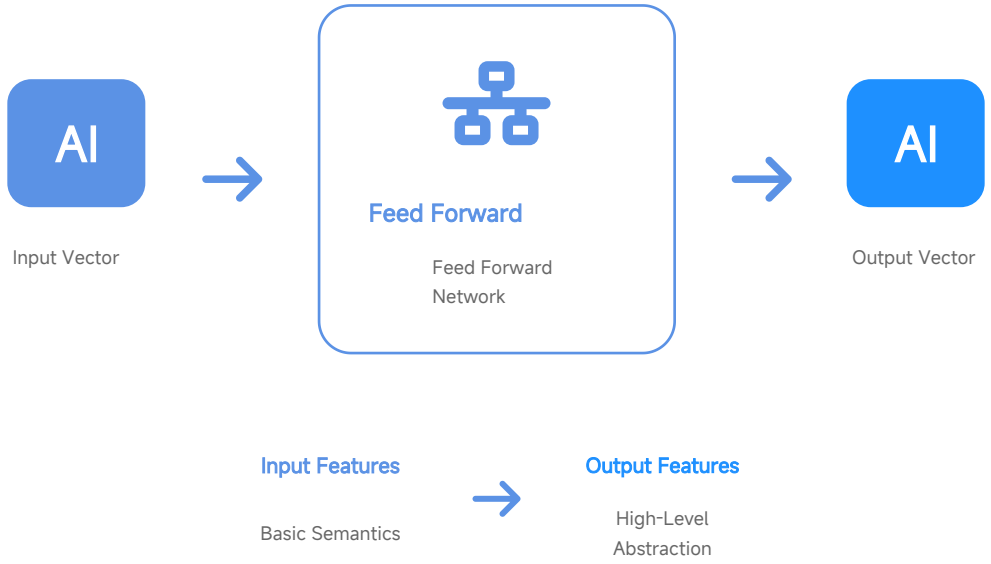
Step 2.1 Multi-Perspective Understanding

Through the Self-Attention mechanism, each word can "see" all other words



📖**Self-Attention** | Words attend to each other; **Multi-Head Attention** | Multiple self-attentions in parallel; **QKV (Query-Key-Value)** | Three core matrices used in the self-attention mechanism to compute attention weights and output

Step 2.2 Deep Processing



Concept Extraction and Abstraction

At this stage, the model performs**independent, deep processing** on each word's representation.

Surface Form:

"What is AI"

Abstract Representation:

"User is asking for the definition of a technical concept"

📖 **Feed Forward Network** | Performs independent nonlinear transformation on each word to extract high-level semantics

Step 2.3 Progressive Layering

Through multi-layer stacking, progressively extract higher-level features



📖**Multi-Layer Stacking |** Stack multiple layers in series to progressively extract higher-level features

Step 2.4 KV Cache Acceleration

Cache historically computed Keys and Values to avoid redundant calculations and significantly increase generation speed

Without KV Cache

Generate 1st word

Compute all history

Generate 2nd word

Re-compute all history

Generate 3rd word

Re-compute again...

With KV Cache

Generate 1st word

Store K,V → Cache

Generate 2nd word

Read cache + Compute new word

Generate 3rd word

Read cache + Compute new word

Higher generation speed

📖**KV Cache (Key-Value Cache)** | Cache Key and Value matrices of historical words to avoid redundant calculations and accelerate generation

Step 3 Prediction: Calculating Probabilities



3.1 Word Selection Strategy

Chain-of-Thought Reasoning





3.2 Technical Implementation

Probability Sampling

Based on the semantic features extracted from previous steps, calculate each candidate word's**probability distribution**, and then select an appropriate word

Step 3.1 Word Selection Strategy

Use Chain of Thought (CoT) to let the model generate intermediate reasoning steps before giving the final answer

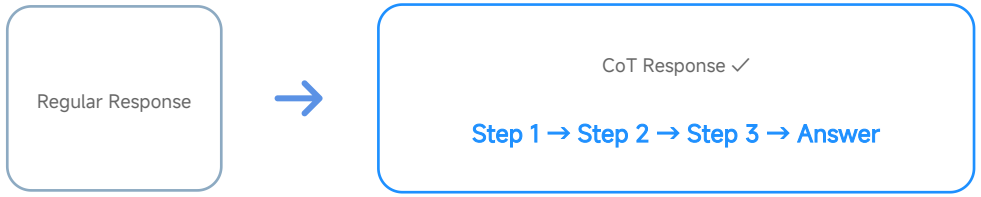
Input "What is AI? Let's think step by step"

Step 1: AI stands for Artificial Intelligence

Step 2: It refers to machines simulating human intelligence

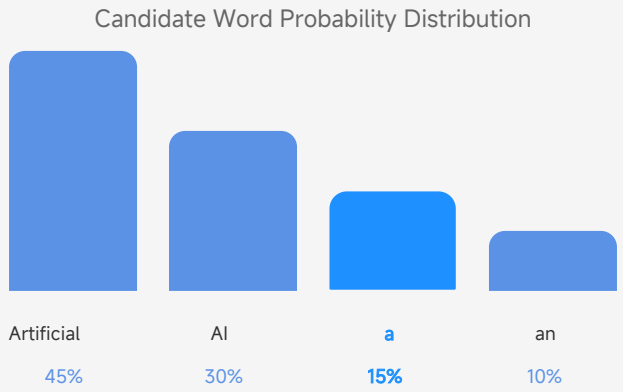
Step 3: This includes learning, reasoning, and problem-solving

Output AI is the simulation of human intelligence in machines...



📖**Chain of Thought (CoT)** | Chain of Thought, letting the model generate intermediate reasoning steps to improve accuracy on complex problems

Step 3.2 Technical Implementation



Softmax

Convert scores to probabilities

Top-k/p Sampling

Control generation diversity

Softmax | A function that converts scores to probabilities; **Top-k/p Sampling** | A sampling strategy that controls generation diversity

Step 4 Repeated Generation: Iterative Loop

1 First Prediction

Input:

"What is AI?"

Prediction Result:

Artificial

2 Second Prediction

Input:

"What is AI? Artificial"

Prediction Result:

Intelligence

Repeat Continuously

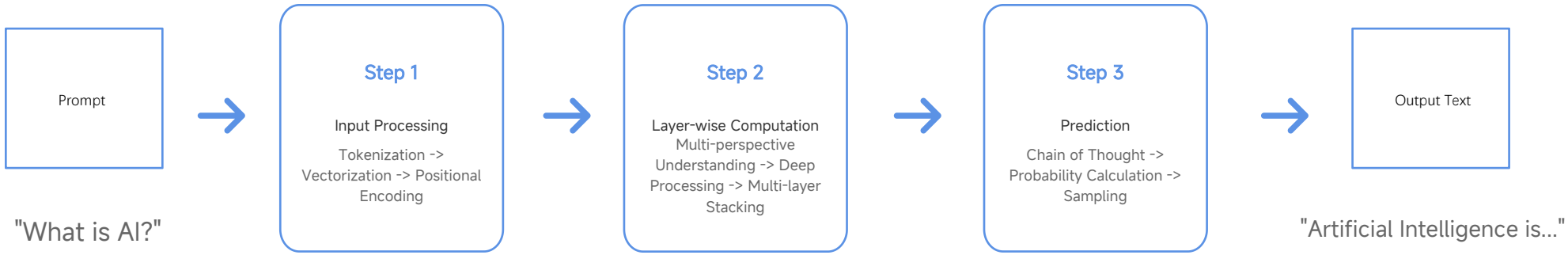
The model **loops** until a complete answer is generated.

Final Output:

"Artificial Intelligence is..."

📖**Autoregressive Generation** | Each time, predict the next word based on already generated content, and iterate continuously

Part 2 Process Summary



Where does the large model's capability come from?

From AI applications we have understood AI's capabilities

From large model workflows we have understood the foundation of AI's powerful capabilities

is the large model's reasoning ability



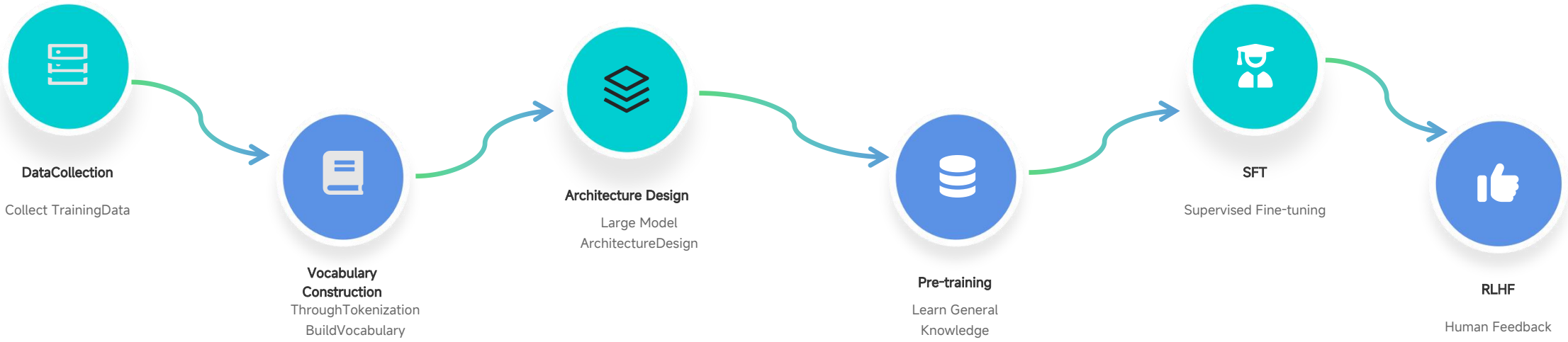
So where does the large model's powerful reasoning capability come from?

PART 3

How to Become an "Omnipotent" AI?

Large Model Training

Overview of Major Steps in Large Model Training



Training a large model is like developing an intern: first learn basic knowledge, then specialized training, and finally adjust based on feedback

Step 1 Data Collection



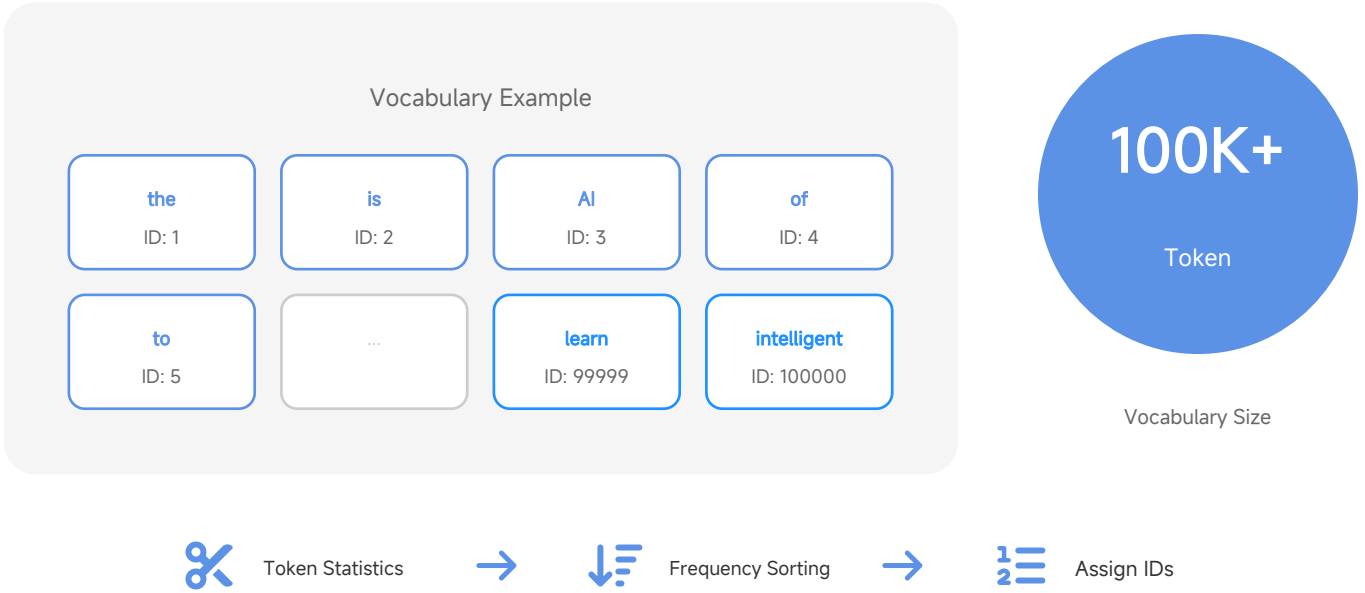
Training Data Scale by Mainstream LLM

Trillions +

Token

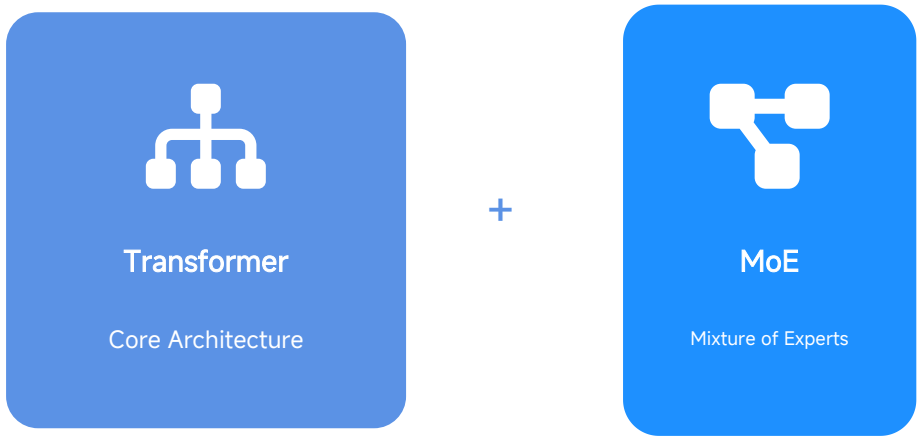
Including web pages, books, papers, code, and other types of text data

Step 2 Build Vocabulary



📖**Vocabulary** | The set of all tokens and their IDs that the model can recognize

Step 3 Architecture Design



Transformer

- Standard architecture for current large language models
- Self-Attention mechanism
- Multi-layer stacking design
- Supports parallel computation

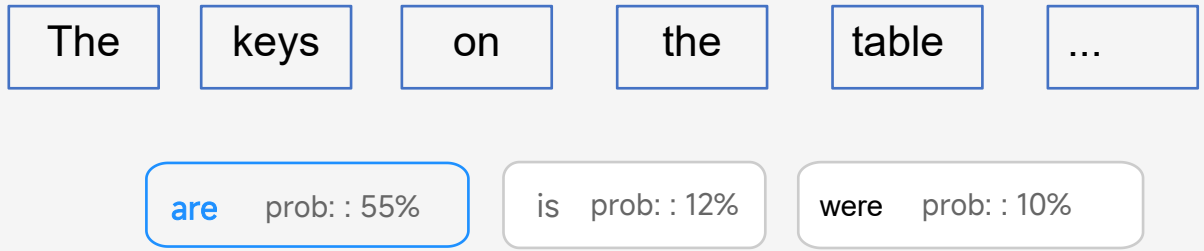
MoE (Optional)

- Mixture of Experts system
- Only activates partial parameters
- Improves computational efficiency
- Supports larger models

📖**MoE (Mixture of Experts)** | Conditional computation architecture, only activates partial parameters

Step 4 Pre-training

Pre-training Task: Predict Next Words



Training Data

Trillion-level

Tokens

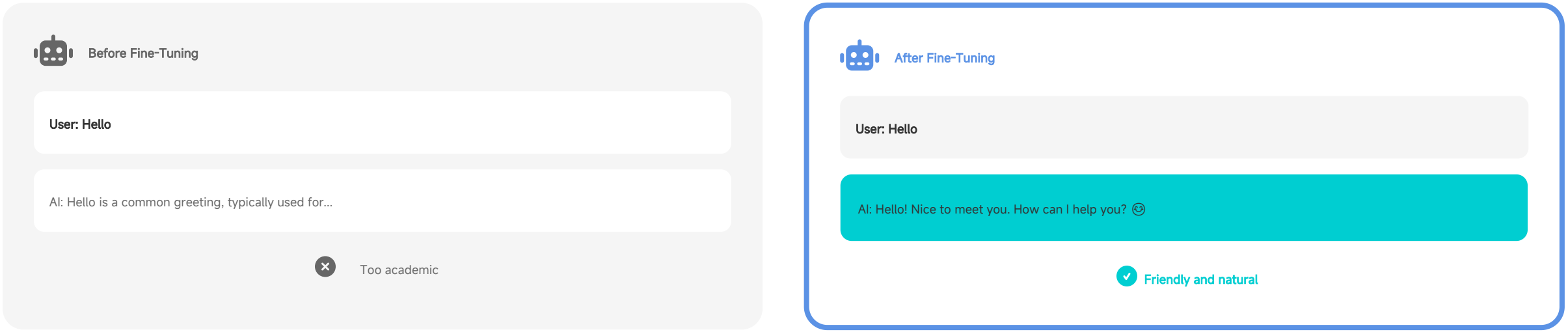
Training Cost

Millions of Dollars

GPU/TPU Computing Power

Pre-training | Training on large-scale unlabeled data to learn general language representations

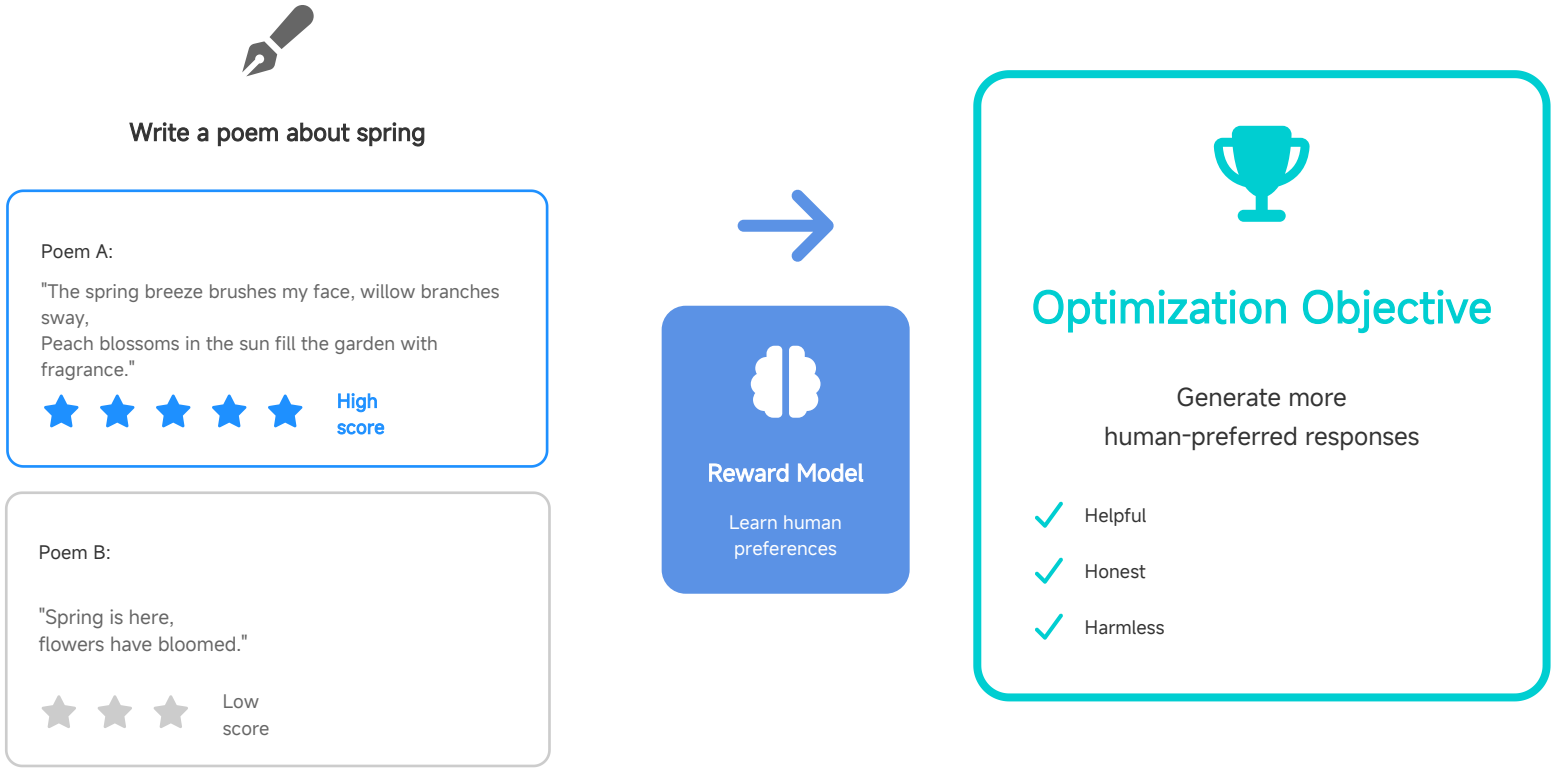
Step 5 Supervised Fine-Tuning



Use high-quality instruction-response pairs to train the model to respond in ways that meet human expectations

📖**SFT (Supervised Fine-Tuning)** | Fine-tune the model using labeled instruction-response data pairs to align it with human intentions

Step 6 RLHF



📖**RLHF (Reinforcement Learning from Human Feedback)** | Reinforcement learning based on human feedback, used to align with human preferences

Chain-of-Thought Capability Development

Training models to learn step-by-step reasoning

1. Understand the Problem: Clarify requirements and given conditions
2. Analyze Steps: Break down the problem into sub-problems
3. Solve Step-by-Step: Address each sub-problem sequentially
4. Verify the Answer: Check the correctness of the result

Pre-training Stage

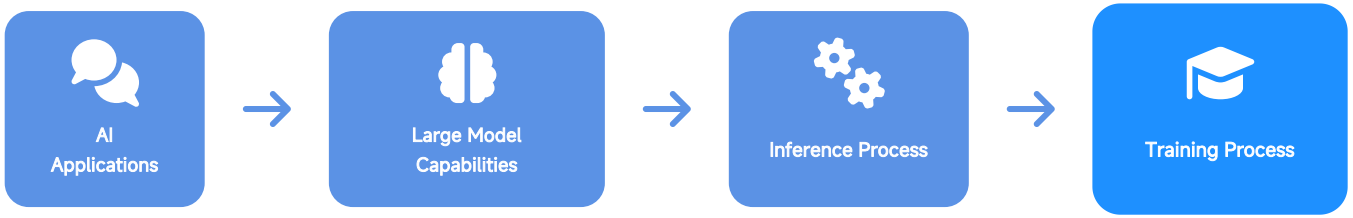
Large-scale reasoning text data

Fine-tuning Stage

Step-by-step annotated data

📖**CoT Training Mechanism** | Through reasoning text in pre-training and step-by-step data in fine-tuning, jointly develop the model step-by-step reasoning capability

Course Summary



Above the Iceberg

Large model applications we see

Below the Iceberg

Computing infrastructure supporting large models

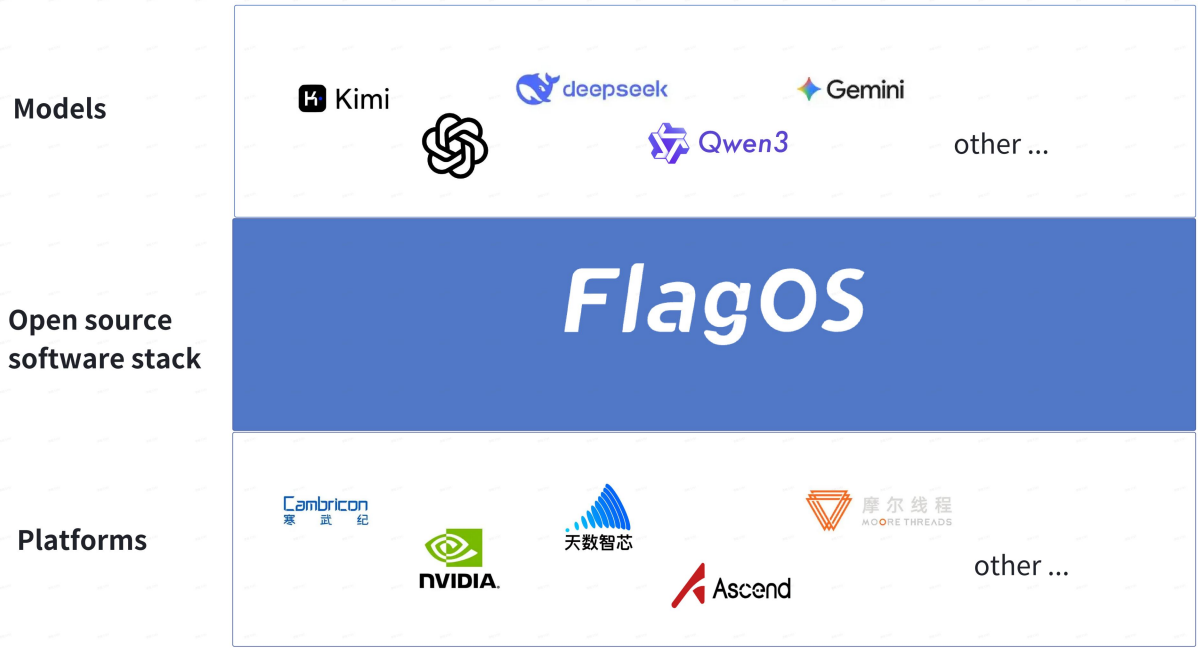
What is supporting large models?

PART 4

Computing Power Behind Large Models

Open Source AI System Software Stack: FlagOS

FlagOS Ecosystem Position



Ecosystem Enablement

Megatron-EL vLLM-plugin-FL TransformerEngine-FL

Core Libraries

FlagGems: High-performance General-purpose AI Operator Library

FlagTree: Unified AI Compiler

FlagScale: Unified Parallel Training and Inference Framework

FlagCX: Unified Communication Library

Tools

KernelGen: AI-powered Operator Development Tool

FlagRelease: Large Model Release Platform

FlagPerf: AI Hardware Evaluation Engine

Reference Resources and Tools

Interactive Understanding of Transformer Architecture

poloclub.github.io/transformer-explainer

Large Model Architecture Visualization

bbycroft.net/llm

Animated Demonstration of Attention Mechanism

prvnsmpth.github.io/animated-transformer

Word Embedding Visualization

https://projector.tensorflow.org/

FlagOS Documentation

docs.flagos.io

Experience How Text is Tokenized

https://tiktokenizer.vercel.app/

Acknowledgments to the following individuals for their support of this course

Cao Zhou

Teng Qiming

Wang Yunbo

Liu Guang

Tao Jian

Shi Lijuan

Chen Jingyi

Liu Yunling

Li Cheng

Zhao Zhiqiang

Jieran Zhang

Zhu Qigang

Jian Chi

PART 5

Hands-on: How to Run Large Models Locally Based on FlagOS